

Concurrent Image Processing Pipeline

Submitted By

M. Rizwan Shafiq CIIT/SP24-BCS-069/ISB

Taimoor Jahangir CIIT/SP24-BCS-073/ISB

BCS-5B

Submitted to

Ma'am Hafiza Sundas

Table of Contents

1.	Introduction	1
2.	Objectives.....	1
3.	Technologies and Tools Used.....	1
4.	Operating System Concepts Used	2
5.	System Architecture	2
6.	Process Roles	2
6.1	Supervisor Process.....	2
6.2	Producer Process.....	3
6.3	Worker Processes	3
6.4	Collector Process	3
7.	Shared Memory Design.....	3
7.1	SharedHeader Structure	4
7.2	Chunk Slot Structure	4
8.	Chunk-Based Image Processing.....	4
9.	Semaphore Synchronization.....	5
9.1	Synchronization Flow	5
Producer		5
Worker.....		5
Collector.....		5
10.	Image Filters	5
10.1	Grayscale Filter.....	6
10.2	Invert Filter.....	6
11.	Pipeline Lifecycle	6
12.	Important Linux System Calls Used	7
13.	Concurrency and Synchronization	7
14.	Error Handling	7
15.	Advantages of the Design	8
16.	Expected Output.....	8
17.	Limitations	8
18.	Conclusion	10
19.	References.....	10

1. Introduction

The purpose of this project is to design and implement a **Concurrent Image Processing Pipeline** using core Operating Systems concepts. The system demonstrates how multiple Linux processes can cooperate through **Inter-Process Communication (IPC)** using **POSIX Shared Memory** and **POSIX Semaphores**. The implementation is written in **C++17** using the **OpenCV** library for image handling.

The project processes a color image concurrently using a pipeline architecture consisting of:

- Supervisor Process
- Producer Process
- Multiple Worker Processes
- Collector Process

The main educational goal is to demonstrate:

- `fork()`
- `waitpid()`
- Shared memory using `shm_open()` and `mmap()`
- Named semaphores
- Producer–consumer synchronization
- Multiprocess coordination
- Shared resource management

2. Objectives

The objectives of the project are:

1. To implement process-based parallelism using Linux system calls.
2. To demonstrate IPC using POSIX shared memory.
3. To synchronize processes using POSIX semaphores.
4. To implement a producer–consumer pipeline architecture.
5. To apply image filters concurrently on image chunks.
6. To understand synchronization problems and their solutions.

3. Technologies and Tools Used

Technology	Purpose
C++17	Core programming language
OpenCV	Image reading and writing
POSIX Shared Memory	Inter-process communication

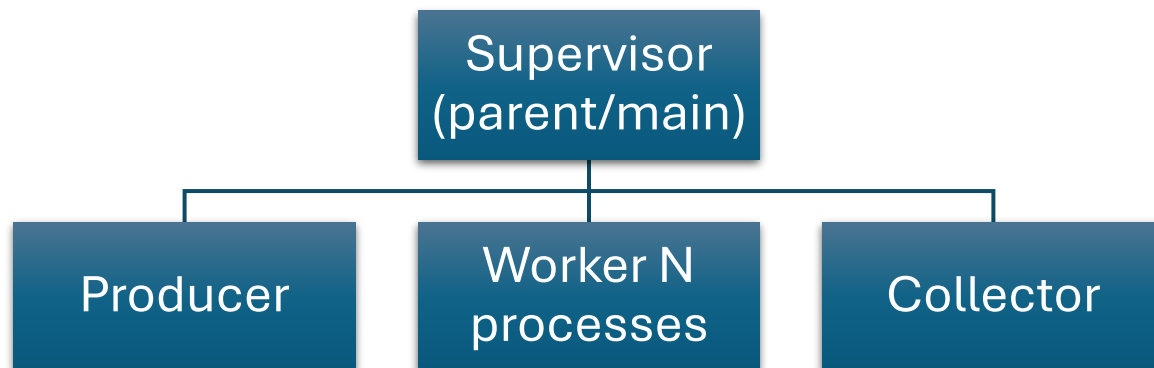
POSIX Semaphores	Synchronization
Linux System Calls	Process management
CMake	Build system
Ubuntu Linux	Development platform

4. Operating System Concepts Used

Concept	API / Mechanism
Process Creation	<code>fork()</code>
Process Waiting	<code>waitpid()</code>
Shared Memory	<code>shm_open()</code> , <code>mmap()</code> , <code>munmap()</code>
Semaphore Synchronization	<code>sem_open()</code> , <code>sem_wait()</code> , <code>sem_post()</code>
Producer–Consumer Problem	Bounded buffer implementation
IPC	Shared memory communication
Concurrent Processing	Multiple worker processes

5. System Architecture

The project follows pipeline-based architecture.



6. Process Roles

6.1 Supervisor Process

The supervisor is the parent process responsible for:

- Creating shared memory
- Creating semaphores
- Initializing shared structures
- Forking all child processes
- Waiting for child completion using `waitpid()`

- Printing performance summary
- Cleaning IPC resources

The supervisor does not process image data directly.

6.2 Producer Process

The producer:

1. Loads the image using OpenCV.
2. Divides the image into row chunks.
3. Waits for an empty slot using `sem_empty`.
4. Copies chunk data into shared memory.
5. Marks slot state as `READY`.
6. Signals workers using `sem_work`.

6.3 Worker Processes

Workers:

1. Wait for available work using `sem_work`.
2. Find a `READY` slot.
3. Process assigned chunks.
4. Apply grayscale or invert filter.
5. Mark slot as `DONE`.
6. Signal collector using `sem_done`.

Chunk assignment uses:

$$\text{chunk_id} \% \text{num_workers} == \text{worker_id}$$

This creates static chunk distribution.

6.4 Collector Process

The collector:

1. Waits for processed chunks using `sem_done`.
2. Copies processed rows into final image.
3. Marks slot as `FREE`.
4. Signals producer using `sem_empty`.
5. Saves final image to disk.

7. Shared Memory Design

The project uses one POSIX shared memory object.

7.1 SharedHeader Structure

The shared header stores:

- Image dimensions
- Number of chunks
- Worker count
- Chunk size
- Pipeline progress
- Worker statistics

```
struct SharedHeader {  
    int width;  
    int height;  
    int channels;  
    int chunk_rows;  
    int total_chunks;  
    int num_slots;  
    int num_workers;  
    int producer_finished;  
    int chunks_done;  
    int filter_type;  
    int worker_stats[MAX_WORKERS];  
};
```

7.2 Chunk Slot Structure

Each slot stores metadata for one image chunk.

```
struct ChunkSlotHeader {  
    volatile int state;  
    int chunk_id;  
    int start_row;  
    int row_count;  
};
```

Each slot contains:

- Slot state (FREE, READY, DONE)
- Chunk identifier
- Starting image row
- Number of rows
- Raw BGR pixel data

8. Chunk-Based Image Processing

The image is divided into horizontal chunks.

Constants:

```
static const int CHUNK_ROWS = 16;  
static const int NUM_SLOTS = 8;  
static const int CHANNELS = 3;
```

Example:

If image height is 480 rows:

```
Chunk 0 → rows 0-15  
Chunk 1 → rows 16-31  
Chunk 2 → rows 32-47  
...
```

Each chunk contains:

16 rows × image width × 3 channels

9. Semaphore Synchronization

Three named semaphores coordinate the pipeline.

Semaphore	Initial Value	Purpose
<code>sem_empty</code>	8	Counts free slots
<code>sem_work</code>	0	Counts ready chunks
<code>sem_done</code>	0	Counts processed chunks

9.1 Synchronization Flow

Producer

```
sem_wait(sem_empty)  
fill slot  
sem_post(sem_work)
```

Worker

```
sem_wait(sem_work)  
process chunk  
sem_post(sem_done)
```

Collector

```
sem_wait(sem_done)  
collect chunk  
sem_post(sem_empty)
```

This creates a cyclic bound buffer synchronization model.

10. Image Filters

The project supports two filters.

10.1 Grayscale Filter

Formula:

$$\text{gray} = (\text{B} + \text{G} + \text{R}) / 3$$

Implementation:

```
px[0] = gray;  
px[1] = gray;  
px[2] = gray;
```

10.2 Invert Filter

Formula:

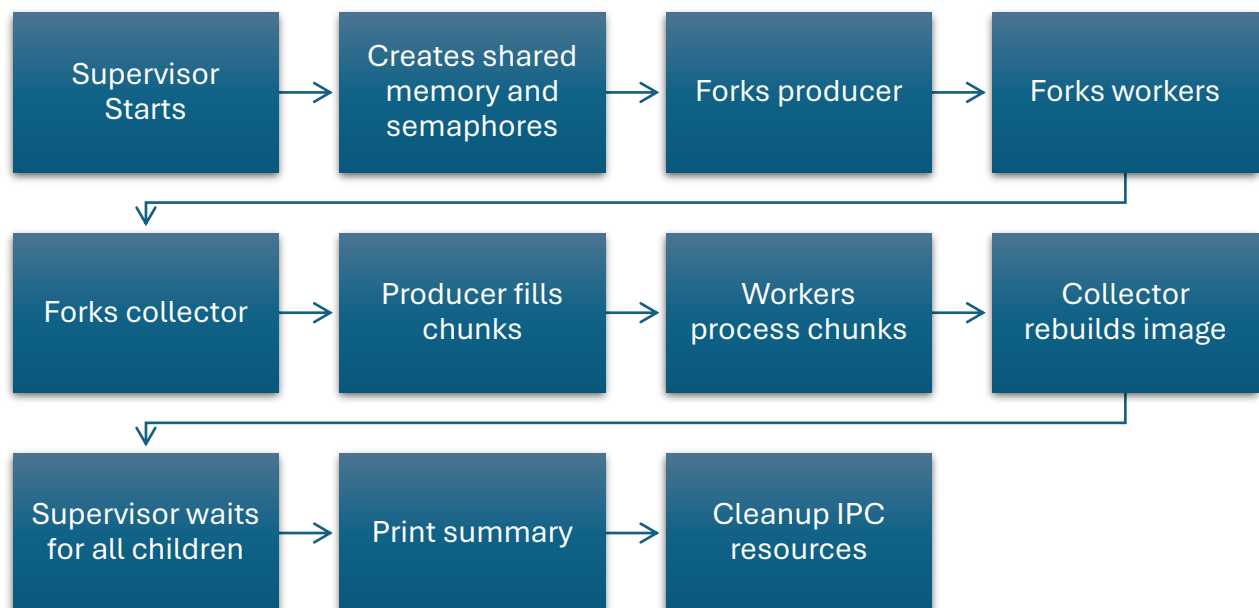
$$\text{new_value} = 255 - \text{old_value}$$

Implementation:

```
px[0] = 255 - px[0];  
px[1] = 255 - px[1];  
px[2] = 255 - px[2];
```

11. Pipeline Lifecycle

Complete execution flow:



12. Important Linux System Calls Used

System Call	Purpose
<code>fork()</code>	Create child processes
<code>waitpid()</code>	Wait for child completion
<code>shm_open()</code>	Create/open shared memory
<code>mmap()</code>	Map shared memory
<code>ftruncate()</code>	Resize shared memory
<code>sem_open()</code>	Create/open semaphores
<code>sem_wait()</code>	Wait on semaphore
<code>sem_post()</code>	Signal semaphore
<code>munmap()</code>	Unmap memory
<code>shm_unlink()</code>	Remove shared memory
<code>sem_unlink()</code>	Remove semaphore

13. Concurrency and Synchronization

The project demonstrates the producer–consumer problem using bounded buffers.

Key synchronization goals:

- Prevent producer from overwriting active slots
- Prevent workers from reading incomplete data
- Prevent collector from collecting unfinished chunks
- Ensure safe chunk reuse

Semaphores ensure proper ordering between processes.

14. Error Handling

The project includes:

- Image loading validation
- Shared memory creation checks
- Semaphore creation checks
- `mmap()` failure checks
- Worker synchronization fallback logic
- Cleanup of stale IPC objects

Example helper:

```
static void die(const char* msg) {
    perror(msg);
    cleanup_ipc(false);
    std::exit(1);
}
```

15. Advantages of the Design

The project intentionally focuses on simplicity and Operating Systems concepts.

Advantages:

- Clear pipeline architecture
- Easy to explain in viva
- Demonstrates real IPC
- Demonstrates process synchronization
- Beginner-friendly design
- Uses actual Linux APIs

16. Limitations

Current limitations include:

- Fixed chunk size
- Static worker assignment
- No dynamic scheduling
- No thread-based parallelism
- No GPU acceleration
- Linear slot scanning

These limitations were intentionally accepted to keep the project educational and OS-focused.

17. How to Build and Run the Project

Project Structure

```
OS_Project_Simple/
├── CMakeLists.txt
├── README.md
├── main.cpp
├── assets/
│   ├── input/      ← place test images here
│   └── output/     ← processed images written here
```

Dependencies (Ubuntu)

```
sudo apt update
sudo apt install -y build-essential cmake pkg-config libopencv-dev
```

Build Instructions

```
cd OS_Project_Simple
cmake -S . -B build
```

```
cmake --build build
Executable: build/image_pipeline
```

Run Instructions

```
./build/image_pipeline <input_image> <output_image> [num_workers] [filter]
```

- `num_workers` is optional (default: **2**, range **1–8**)
- `filter` is optional: **grayscale** (default) or **invert**

Example Commands

```
# Copy a test image into assets/input first
cp ~/Pictures/photo.png assets/input/photo.png

./build/image_pipeline assets/input/photo.png assets/output/output.png
./build/image_pipeline assets/input/photo.png assets/output/output.png 4
./build/image_pipeline assets/input/photo.png assets/output/output.png invert
./build/image_pipeline assets/input/photo.png assets/output/output.png 4 invert
```

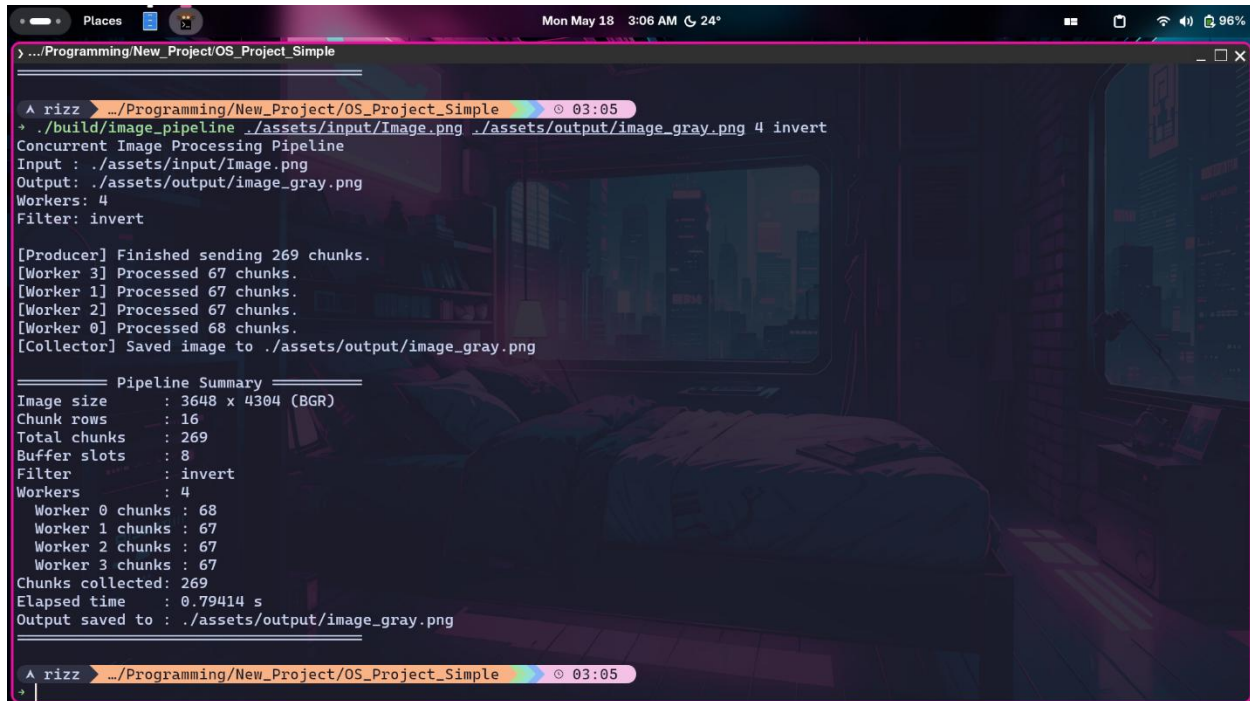
18. Expected Output

Example console output:

```
Concurrent Image Processing Pipeline
Input : assets/input/photo.png
Output: assets/output/output.png
Workers: 4
Filter: grayscale

[Producer] Finished sending 48 chunks.
[Worker 0] Processed 12 chunks.
[Worker 1] Processed 12 chunks.
[Worker 2] Processed 12 chunks.
[Worker 3] Processed 12 chunks.
[Collector] Saved image to assets/output/output.png

===== Pipeline Summary =====
Image size      : 640 x 480 (BGR)
Chunk rows      : 16
Total chunks    : 30
Buffer slots    : 8
Filter          : grayscale
Workers         : 4
  Worker 0 chunks : 8
  Worker 1 chunks : 8
  Worker 2 chunks : 7
  Worker 3 chunks : 7
Chunks collected: 30
Elapsed time    : 0.0523 s
Output saved to : assets/output/output.png
=====
```



```
.../Programming/New_Project/OS_Project_Simple
^ rizz ~./Programming/New_Project/OS_Project_Simple 03:05
+ ./build/image_pipeline ./assets/input/Image.png ./assets/output/image_gray.png 4 invert
Concurrent Image Processing Pipeline
Input : ./assets/input/Image.png
Output: ./assets/output/image_gray.png
Workers: 4
Filter: invert

[Producer] Finished sending 269 chunks.
[Worker 3] Processed 67 chunks.
[Worker 1] Processed 67 chunks.
[Worker 2] Processed 67 chunks.
[Worker 0] Processed 68 chunks.
[Collector] Saved image to ./assets/output/image_gray.png

===== Pipeline Summary =====
Image size      : 3648 x 4304 (BGR)
Chunk rows     : 16
Total chunks    : 269
Buffer slots    : 8
Filter         : invert
Workers        : 4
  Worker 0 chunks : 68
  Worker 1 chunks : 67
  Worker 2 chunks : 67
  Worker 3 chunks : 67
Chunks collected: 269
Elapsed time    : 0.79414 s
Output saved to : ./assets/output/image_gray.png

^ rizz ~./Programming/New_Project/OS_Project_Simple 03:05
+ |
```

19. Conclusion

This project successfully demonstrates how multiple Linux processes can cooperate using shared memory and semaphores to build a concurrent image-processing pipeline. The implementation combines important Operating Systems concepts such as IPC, process synchronization, bounded-buffer producer-consumer design, and process coordination.

The project achieves concurrent image processing while maintaining a beginner-friendly and viva-oriented architecture. It serves as a practical demonstration of how Operating Systems concepts are applied in real multi-process applications.

20. References

1. OpenCV Documentation
2. Linux Man Pages (`fork`, `mmap`, `shm_open`, `sem_open`)
3. Operating System Concepts – Silberschatz
4. POSIX IPC Documentation
5. Ubuntu Linux Documentation